Tessera

System architecture and feature review

Executive narrative

Tessera is a durable research-control workspace, not a generic agent shell or a compute scheduler. Its intended value is a traceable path from a goal through a bounded experiment and evidence to a human decision. The repository implements meaningful slices of that path: local control records, immutable plans, scoped credentials, execution admission, durable ledgers, a sandboxed desktop shell, CPU supervision, artifact quarantine, recovery, and narrowly fenced coordinator and Kubernetes seams. The review finds a deliberately conservative architecture: authority is centered in the control plane; the desktop, Astra coordinator, OpenCode, workers, and Kubernetes adapters are clients or constrained effectors. This is a strong shape for honest research records, but it is not yet a complete pilot.

Bottom line

Implemented components create, retain, and display bounded records. Synthetic and opt-in fixtures exercise selected seams. Real provider execution, supported remote control, selected OpenCode compatibility, Kubernetes target attachment, artifact release, result acceptance, and scientific validation remain absent or explicitly gated. No test count or component pass is treated here as a live experiment, provider-billing, laptop-independence, or scientific-success claim.

Snapshot and evidence caveat

Review date: 14 September 2026. Repository snapshot: main at 6668ab8, before this review artifact. The working tree was intentionally dirty: 65 tracked paths modified, 27 untracked paths, 92 entries total; no staged paths. The uncommitted Astra, remote-workflow, provisioner, script, desktop, control, worker, documentation, and test work was inspected as repository content, not represented as merged. Current source constant: schema v43 in src/tessera_control/migrations.py. README and implementation-progress text still describe the historical schema-v42 evidence checkpoint; those claims are preserved as dated historical evidence, not silently upgraded. Schema v43 adds operation-bound remote workflow lifecycle records for new packages only: heartbeat, opaque checkpoint, reconnect/outage observations. It does not prove a run, provider termination, replacement submission, held-restore writability, or laptop independence.

Scope and method

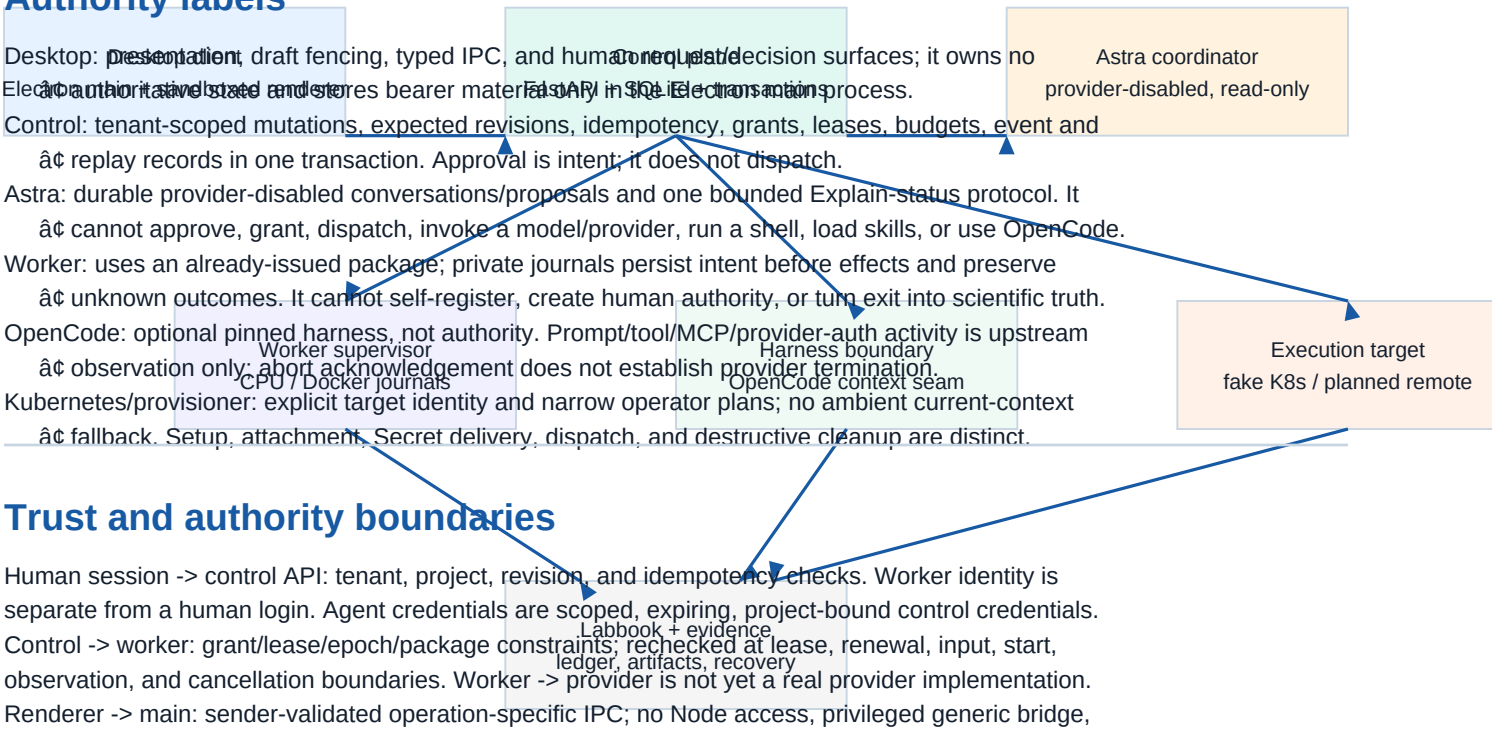
Read DESIGN.md, README.md, CONTRIBUTING.md, decisions, major control/worker/provisioner/agent and desktop packages, migrations/recovery, Kubernetes/OpenCode/live-E2E/setup/conformance scripts, tests, and the current diff. This is an architecture review of source and documented evidence; no screenshots, secrets, credentials, application-code edits, provider calls, Docker/Kubernetes live runs, or commits.

Architecture: components and authority

The control plane is the authoritative record and admission boundary. Everything else is fenced by it.

Authority labels

Desktop: presentation, draft fencing, typed IPC, and human request/decision surfaces; it owns no
  â¢ authoritative state and stores bearer material only in the Electron main process.
Control: tenant-scoped mutations, expected revisions, idempotency, grants, leases, budgets, event and
  â¢ replay records in one transaction. Approval is intent; it does not dispatch.
Astra: durable provider-disabled conversations/proposals and one bounded Explain-status protocol. It
  â¢ cannot approve, grant, dispatch, invoke a model/provider, run a shell, load skills, or use OpenCode.
Worker: uses an already-issued package; private journals persist intent before effects and preserve
  â¢ unknown outcomes. It cannot self-register, create human authority, or turn exit into scientific truth.
OpenCode: optional pinned harness, not authority. Prompt/tool/MCP/provider-auth activity is upstream
  â¢ observation only; abort acknowledgement does not establish provider termination.
Kubernetes/provisioner: explicit target identity and narrow operator plans; no ambient current-context
  â¢ fallback. Setup, attachment, Secret delivery, dispatch, and destructive cleanup are distinct.



Trust and authority boundaries

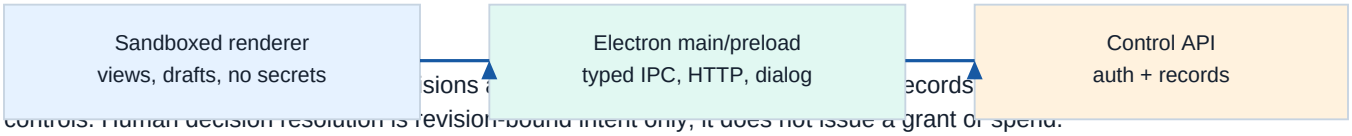
Human session -> control API: tenant, project, revision, and idempotency checks. Worker identity is separate from a human login. Agent credentials are scoped, expiring, project-bound control credentials. Control -> worker: grant/lease/epoch/package constraints, rechecked at lease, renewal, input, start, observation, and cancellation boundaries. Worker -> provider is not yet a real provider implementation. Renderer -> main: sender-validated operation-specific IPC; no Node access, privileged generic bridge, or renderer-held secrets. Artifacts and external inputs are untrusted.

Desktop and UI flow

The desktop is a safe client of an independently running control plane, even when co-located.

Connection and read flow

â¢ 1. A named connection supplies endpoint, identity, organization, compatibility, and health context.
â¢ 2. Electron main holds the credential in memory and performs HTTP. The sandboxed renderer uses narrow IPC and clears in-flight reads, drafts, cached context, and pending commands on connection/project changes so a stale response cannot cross a connection boundary.
â¢ 3. The UI reads projects, plans, decisions, conversations, labbook ledgers, experiments, evidence, and recovered history. Held recovery is visibly read-only; browsing never starts work.



controls. Human decision resolution is revision-bound intent only, it does not issue a grant or spend.
Conversation: ordinary interaction threads preserve sender, intent, context, and delivery status.
Coordinator workbench: Discuss, Draft experiment, Explain status, and Review results modes create typed, provider-disabled Astra turns. Selected task/experiment revision chips are explicit. The UI does not expose raw harness/session identifiers and disables command controls when stale/offline/held.
Research ledger: create studies, register variants, record explicitly manual/imported outcomes, assess sources, make candidate findings, accept/reaffirm/narrow/supersede/retract with human action, and export a complete bounded study JSON envelope. These are evidence and judgment records, not automatic science.

Desktop authority diagram

Design-aligned guardrails

Renderer isolation, restrictive CSP, no remote navigation, typed sender validation, and explicit native file picker/save dialogs are present. Source URIs are displayed as text; the renderer does not fetch or
â¢ open external content. Complete-ledger rendering rejects incomplete or inconsistent envelopes.
Closing Electron does not stop the backend. The desired design is that remote authorized work can continue under bounded disconnect policy; current desktop/control fixtures demonstrate local synthetic paths, not
â¢ a completed remote/laptop-independent workflow.

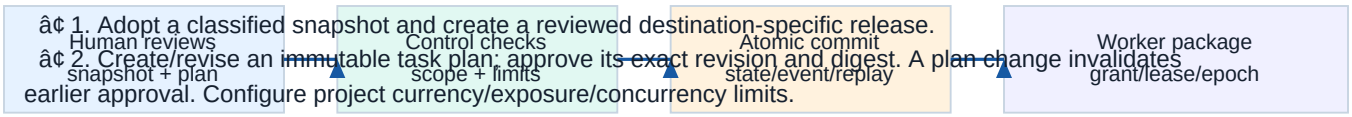
Evidence

desktop/README.md; desktop/main.mjs; desktop/preload.cjs; desktop/renderer.mjs; desktop/connection.mjs; desktop/test/*.mjs and *.cjs; scripts/check_desktop_control.py. Historical v42 checkpoint records 86 desktop Node tests and a native synthetic Electron/control fixture, including held-restore browsing.

Control-plane transaction flow

The most mature architectural idea is that durable state, audit event, and idempotency response commit together.

Admission sequence



â¢ 1. Adopt a classified snapshot and create a reviewed destination-specific release.

â¢ 2. Create/revise an immutable task plan; approve its exact revision and digest. A plan change invalidates earlier approval. Configure project currency/exposure/concurrency limits.

â¢ 3. A human issues a grant for the exact task, worker, snapshot, release, pinned image, budget, expiry, and explicit acknowledgement of unmeasured local infrastructure. The worker heartbeats and acquires one short lease with a hard deadline; epochs fence stale/revoked work.

â¢ 4. Before an executable local CPU package returns, external-operation intent records stable operation and Docker lookup identities, package/request digests, profile/adapter versions, desired state, and the manual-reconciliation retry class. A replay returns the same identity, not a second submit.

Transaction and state semantics

Store._mutate performs authorization, revision/response consistency, state mutation, ordered event, and idempotency response in one SQLite transaction. Reuse of a key with different content conflicts. Reads are tenant/project scoped; list bounds are enforced. Consumer/outbox behavior is designed for dedupe. Grant revoke changes desired state to cancel_requested and advances the epoch; it is not a provider call. Current reconciliation combines attributed worker and fake-receipt observations by observed time. A later or equal-time unknown/running/unconfirmed observation blocks terminal reconciliation and preserves exposure. The data model deliberately separates session, experiment, attempt, external operation, artifact/measurement, and conclusion lifetimes. A completion, receipt, or evidence hash cannot imply a scientific conclusion.

What is and is not implemented

Implemented bounded control records: projects, credentials, plans/revisions, approvals, input releases, limits, grants, leases, operation intents, observations, research ledger, interaction, artifacts, and recovery. ExecutionStore explicitly exposes execution_dispatched=false for its base local records. Not complete: real provider submission/accounting, production identity/organization login, PostgreSQL or multi-writer deployment, universal cost caps, full outbox consumers, and an accepted pilot workflow.

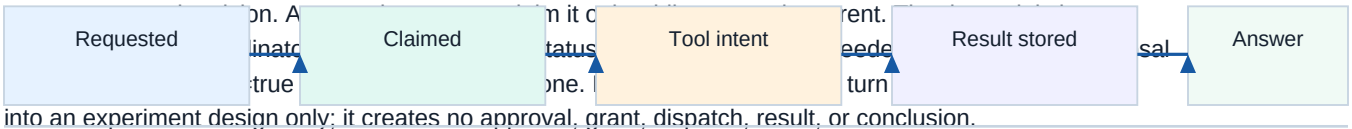
Evidence

src/tessera_control/store.py, api.py, execution.py, execution_api.py, external_operations.py, runtime_packages.py, input_delivery.py, workers.py, models.py; tests/test_execution*.py, test_plans.py, test_fake_provider_dispatch.py, test_integrated_experiment_workflow.py.

Astra coordinator and agent status flow

Astra is intentionally a durable, provider-disabled coordination surface rather than an autonomous executor.

Coordinator turns

A human creates an Astra project conversation turn with immutable selected task/experiment context and a


into an experiment design only; it creates no approval, grant, dispatch, result, or conclusion.

Explain-status: a one-tool, read-only state machine

Schema v42 introduced this narrow protocol. A human selects one immutable experiment design revision. A scoped agent claims a short binding, persists a tool intent, and can invoke exactly read_experiment_status with exactly {identifier: experiment_id}. The facade is GET-only, project-scoped, strict, bounded, and allowlisted. It returns control-record operation status and scientific_validity=not_assessed.
The result is compared to current control data, stored with argument/result digests, observed time and deterministic provider-disabled provenance, then rendered as an attributed answer. If acknowledgement is lost after intent, the persisted intent prevents a second facade invocation. Expiry, cancellation, stale design context, wrong credential/agent, or changed binding fail closed.

Authority boundary

The agent facade forbids approve, grant, dispatch, provision, secret, shell, accept_result, publish, and skill loading. It has no generic HTTP method, control-store import, provider/model client, harness, MCP, or runtime-skill loader. Explain-status is operational description only: it neither mutates experiment state nor converts an observed operation into provider success or scientific validity.

Current-schema addition

Schema v43 adds remote_workflow_runs/checkpoints adjacent to an already-issued operation. Worker heartbeat, checkpoint, and reconnect/outage records are fenced to the original worker/lease/operation/epoch and are operational_only. They cannot fabricate a legacy run, replace a submission, widen authority, or make a held restore writable. The synthetic reconnect fixture retains unknown/reconciliation-required state.

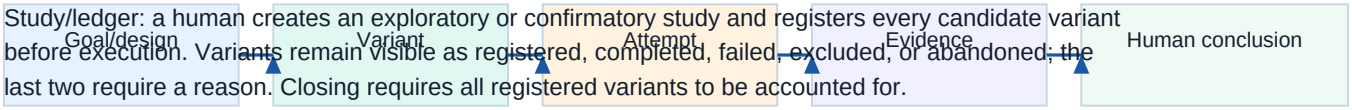
Evidence

src/tessera_control/astra*.py, remote_workflows.py; src/tessera_agent/coordinator.py, facade.py; tests/test_astra_*.py, test_remote_workflows.py; scripts/check_remote_reconnect_synthetic_e2e.py.

Experiment and labbook lifecycle

The research model preserves evidence, interpretation, and human conclusions as different records.

Lifecycle



Study/ledger: a human creates an exploratory or confirmatory study and registers every candidate variant before execution. Variants remain visible as registered, completed, failed, excluded, or abandoned; the last two require a reason. Closing requires all registered variants to be accounted for.

Evidence: sources include quoted passage, content digest, locator, assessment status, and correction lineage. Findings are candidates until an eligible human accepts supported citations and appropriate study state.

Review: finding reaffirmation, narrowing, supersession, and terminal retraction append history. Corrections preserve source versions and mark dependent findings for review rather than silently rewriting history.

Goal-to-decision: immutable experiment design revisions carry goal/success criterion, baseline/intervention, attempt provenance, actual custody links, human-reported observation distinction, and reviewed conclusion lineage. Operational succeeded and human conclusion cards remain labelled scientific_validity=not_assessed.

Artifact and claim separation

An execution attempt may link an operation and/or explicitly human-reported observation. A provider receipt does not validate an artifact; a validated artifact does not accept a result; a result-bookkeeping entry does not create a scientific finding. Export is a bounded complete ledger projection, not a reproducible bundle of code, environments, released bytes, reports, or universally valid claims.

Labbook integrity properties

The labbook is a permission-filtered projection of control transitions and versioned evidence, not a second authority. It includes record IDs, actor/time/revision/provenance/access labels. Historical views are read-only. Design calls for explicit disclosure of incomplete/restricted history and projection lag. Synthetic integrated coverage deliberately retains all 80 variants rather than selecting favorable results.

Evidence

src/tessera_control/research.py, research_models.py, experiments_*.py, result_acceptances.py; desktop/renderer.mjs; docs/records-api.md; tests/test_research.py, test_goal_to_decision.py, test_finding_*.py, test_integrated_experiment_workflow.py.

CPU and Kubernetes execution flow

Execution is controlled as a bounded, recoverable operation; live provider execution is not enabled.

Local CPU development profile

development-local-cpu-v1 is an opt-in native one-shot worker profile around a control-issued local-cpu-v1 package. It validates a pinned Docker identity, read-only released inputs, private worker credential, grant, lease and epoch. CpuSupervisor owns a private journal and persists start intent before Docker start. On lost start/observe/cancel reporting it marks the operation uncertain and reconciles the same identity; it does not issue a replacement start. It rechecks authority and bounded duration and records cancellation as requested/unresolved/confirmed only when a matching termination observation confirms it.

CPU safety posture

The fixture profile uses non-root execution, read-only root, no network, bounded CPU/RAM/scratch/processes, pinned image, no host mounts, no automatic restart/pull, and read-only input mounts. It is development evidence, not a supported persistent-output or research-pilot backend.

Kubernetes seams

KubernetesExecutionAdapter accepts only an admitted, provider-disabled package and injected callbacks. It pins package digest, operation/attempt/lease/epoch, image, workload, target cluster/namespace/PVC identity, and network-policy evidence. It persists local intent before a deterministic Job create; lost replies use lookup/reconciliation, never a second Job. Cancellation preserves Job UID and requires fresh Job/owned-Pod absence to establish termination evidence. Collection handoff emits metadata only; it does not move bytes. KubernetesDispatchWorkerService consumes a human-approved v36 outbox claim after complete active-grant predicate rechecks. It durably journals claim, descriptor, reports, and actions, observes/cancels existing work after lost authority, and never uses Kubernetes credentials/target discovery itself.

Operator setup and conformance

Astra operator setup v2 is a separate, explicit Docker Desktop setup boundary. It requires an absolute kubeconfig and docker-desktop context and applies only a retained Namespace, deny-all NetworkPolicy, and 64 MiB PVC after explicit plan-digest approval. It does not attach a target, run a workload, create a Secret, enroll a worker, activate a provider, or delete resources.

The opt-in quota/network conformance probe creates a separate run-labelled namespace/PVC/pods/policies. It requires declared quota annotations and records actual ENOSPC/EDQUOT evidence or reports unsupported.

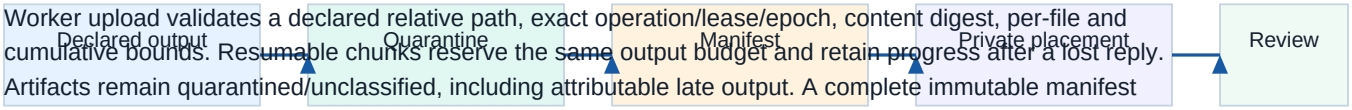
Evidence

src/tessera_worker/cpu.py, supervisor.py, development_runtime.py, kubernetes_*.py, docker_desktop_kubernetes.py; src/tessera_provisioner/astra_setup.py; docs/astra-operator-setup.md; scripts/check_*kubernetes*.py; tests/test_cpu.py, test_supervisor.py, test_kubernetes*.py, test_astra_setup.py.

Artifact custody, persistence, and recovery

Tessera has durable custody and quarantine mechanics; it deliberately does not equate custody with release.

Output path



Worker upload validates a declared relative path, exact operation/lease/epoch, content digest, per-file and cumulative bounds. Resumable chunks reserve the same output budget and retain progress after a lost reply. Artifacts remain quarantined/unclassified, including attributable late output. A complete immutable manifest requires exactly the approved output set and rechecks retained bytes.

Development output storage reserves userspace bytes/inodes in a private marked workspace, seals only declared safe nodes after fresh matching confirmed termination, transfers the sealed set, and cleans only after durable project placement. This is a synthetic/development seam, not the required Linux enforcement backend.

Private project persistence

The local filesystem backend receives bytes from control, not worker paths or storage credentials. It uses hashed tenant/project/object namespaces, no-overwrite stage/link publication, reconciliation, checksums, and retention metadata. Placement is not classification, public download, result acceptance, cost settlement, or provider/scientific success. S3/IAM/KMS, public URLs, external export/move, and production activation remain unimplemented. Deletion is manual and fail-closed for held/referenced/checkpointed/accepted custody.

Recovery

Backup uses SQLite online copy plus a format-v2 object bundle when committed filesystem objects exist. It validates exact trusted schema/triggers, foreign keys, object identity/checksum, bounded size/object/time limits, and publishes a manifest. Restore rejects unsafe/altered input, creates a new owner, revokes all copied credentials, and enters permanent held_restore. Reads and desktop browsing are allowed; all mutations, dispatch, renewals, replay, cleanup, target bootstrap, Secret materialization, and provider reconciliation are refused. Original service/workers/jobs are not stopped or fenced by restore.

Open work

Automatic container output collection awaits verified persistent Linux storage with hard byte/inode quota, mount/custody/noexec constraints, container exhaustion, exit, restart, and replay evidence. macOS Docker Desktop was explicitly found insufficient for the required noexec output profile.

Environment profiles, offline behavior, cancellation, and

Profiles are intentionally explicit so ambient machine state cannot become research authority.

Environment/profile matrix

â¢ Local developer: supported development workflow is Python 3.11+, uv.lock and Docker Compose/native fixtures. Compose publishes control only to 127.0.0.1; test has no network. This is not a deployment claim.

â¢ Development local CPU: opt-in, pinned image, synthetic fake provider and quarantine-only outputs.

â¢ OpenCode: source-inspected v1.18.30 context contract and in-process/HTTP-SSE seams; no selected real server or recorded compatibility result. Real fixture is opt-in and requires exact executable/version/schema pins.

â¢ Kubernetes: fake-client contracts and opt-in Docker Desktop fixtures use explicit target facts; no supported target attachment, live Kubernetes client path, credential delivery, OpenCode activation, or result acceptance.

â¢ Remote OAuth fixtures: v1 one-call and v2 ten-turn scripts are default-disabled retained operational harnesses. They require a pre-existing remote runner/deployment, opaque owner-bound runtime reference, explicit preflight, digest-bound confirmation, exact image/target, bounded calls/tokens/time, and no API-key/fallback/top-up path.

Offline, reconnect, and cancellation

The design requires desktop disconnection to leave independent backend and eligible work running under bounded policy; local source contains a remote-workflow lifecycle and a two-process synthetic reconnect fixture only.
If the control plane is unreachable, workers do not expand grants and preserve unknown state. Reconnect binds to the same operation, lease, epoch and checkpoint sequence. Unknown control/provider effects block unsafe replacement. Offline desktop edits are drafts, never approvals or assumed cancellations.
Cancellation distinguishes desired request, delivery, action fence, writer quiescence, provider termination, credential expiry/revocation, workspace, and charge reconciliation. Current projections only establish request, unresolved/unknown, or worker-confirmed termination from attributed observations. No green stop badge is justified by an abort acknowledgement or local terminal state alone.

Security posture and gaps

Implemented/explicit: tenant checks at access; scoped expiring credentials; separate human/agent/worker identities; revision/epoch fences; loopback local service; browser-origin rejection; hashed private object names; no raw secrets in records/events/diagnostics; opaque metadata-only credential leases and fake handles; private journals; input/output path checks; renderer isolation; untrusted artifact treatment; held recovery.
Missing/not proven: production organization authentication/rotation, cryptographic worker build trust and capability attestation, production secret backend/delivery, selected TLS remote control deployment, provider cost/termination reconciliation, signed desktop/package updates, storage enforcement, and production monitoring.

Testing and evidence matrix

Status labels describe observed scope, not product readiness.

Area	Evidence present	What it does NOT prove
Control/store/migrations	synthetic unit/integration suites	remote/provider/pilot completion
Desktop	Node + native synthetic fixture	visual or production desktop QA
CPU worker/supervisor	Docker isolation/cancel fixtures	persistent pilot backend
OpenCode	in-process + HTTP/SSE seams	real compatible server/provider
Kubernetes	fake-client + opt-in contracts	supported attached target
Astra + remote workflow	provider-disabled deterministic paths	model run/laptop independence
Recovery/artifacts	held restore + custody tests	release/result/scientific truth
Live/provider	opt-in harnesses implemented	a run or billing evidence

Recorded historical evidence

docs/implementation-progress.md records a reproducible schema-v42 baseline dated 2026-09-13: 848 Python tests passed natively, 37 focused integrated/Astra/migration/recovery tests without skips, 86 desktop Node tests, Ruff lint/format, Python syntax compilation, and a native synthetic desktop/control fixture with execution_started=false. It also preserves the 2026-09-09 historical schema-v24 collection: 586 Python tests and narrower fixture results. Those are dated repository assertions, not commands rerun for this review.

This review validation

Inspected source and dirty diff; identified schema v43 from code; used installed Ghostscript ps2pdf to render this vector/text PDF. No repository runtime test or opt-in Docker/Kubernetes/OpenCode/provider fixture was executed because the request was document production, not live operational validation.

Implemented, synthetic, opt-in, planned

The distinction is the primary finding: preservation of boundaries is stronger than the current delivery surface.

Implemented bounded features

Local FastAPI/SQLite control foundation; explicit migrations through v43; transactionally recorded project, plan, approval, grant, lease, operation, event and idempotency state; scoped credentials; research ledger; artifact quarantine/resume/manifest/disposition/placement; held recovery; desktop shell; CPU supervisor; Astra durable provider-disabled coordinator/status records; remote-workflow operational journal; and narrow operator setup contract. These features remain partial against their DESIGN.md workflow contracts.

Synthetic / provider-disabled

Fake provider receipts; OpenCode-shaped in-process server and exact-schema HTTP/SSE seam; provider-disabled Astra status call; fake-client Kubernetes Job/PVC dispatch and integrated baseline/intervention workflow; temporary loopback control, Docker, Electron, and reconnect fixtures. Their records deliberately retain not_assessed/not_established labels and do not become provider or scientific evidence.

Opt-in / live operational harnesses

Native Docker CPU/runtime/development-worker and combined synthetic fixtures; native desktop fixture; real OpenCode compatibility fixture only with exact pin; Docker Desktop Kubernetes conformance/dispatch checks; Astra quota/network probe; remote Kubernetes OAuth v1 and v2 runners. Harness existence is not a recorded run. Live OAuth docs state the repo has no enrolled runner, remote credential bridge, target, or confirmation.

Planned or blocked

One real public/simple CPU baseline plus intervention that survives desktop disconnect; remotely reachable control plane and execution; selected/enrolled OpenCode node; real provider adapter/accounting; credential delivery; Kubernetes target attachment; hard quota/restart output evidence; artifact release and accepted scientific result policy; production organization identity/hosting; portable reproduction bundles; monitoring.

Concrete next steps

â¢ 1. Keep the current v43 source/documentation schema mismatch explicit; update the status matrix only with a dated, reproducible v43 checkpoint rather than relabelling v42 results.

â¢ 2. Select one real remote CPU target and a small public workflow. Exercise disconnect, lost reply, revoke, restart, cancellation, all four outcome labels, custody, review, and effort measurement end-to-end.

â¢ 3. Meet OpenCode/provider activation gates with exact server/build/schema pins, enrolled-node trust, TLS, provider-specific lookup/usage/termination reconciliation, and no credential or billing fallback.

â¢ 4. Establish a supported persistent-output backend with real Linux byte/inode exhaustion and restart evidence.

â¢ 5. Treat v43 remote lifecycle as operational evidence until an independently observed remote run proves the advertised continuation properties; retain unknown and reconciliation-required outcomes.

Source map and review limitations

Selected paths support the claims in this review; the complete source tree and dirty changes were inspected.

Primary contracts

DESIGN.md sections 2-9; README.md; CONTRIBUTING.md; docs/implementation-progress.md; docs/compatibility-matrix.md; docs/execution-admission.md; docs/recovery.md; docs/artifact-quarantine.md; desktop/README.md.

Decisions

docs/decisions/0001-control-plane-foundation.md through 0012-credential-delivery-boundary.md, especially 0003 persistent output storage, 0004 OpenCode harness boundary, 0008 project artifact storage, 0009 Kubernetes conformance, 0010 adapter composition, 0011 provisioning, and 0012 credential delivery.

Control and agent

src/tessera_control/store.py, migrations.py, api.py, execution.py, external_operations.py, research.py, interaction.py, artifacts.py, project_artifacts.py, recovery.py, astra.py, astra_status.py, remote_workflows.py; src/tessera_agent/facade.py and coordinator.py.

Worker and provisioner

src/tessera_worker/supervisor.py, cpu.py, development_runtime.py, runtime.py, service.py, outputs.py, sealed_outputs.py, opencode_harness.py, opencode_http.py, opencode_live_adapter.py, kubernetes_contracts.py, kubernetes_execution.py, kubernetes_dispatch_service.py, docker_desktop_kubernetes.py; and src/tessera_provisioner/astra_setup.py.

Operational scripts and tests

scripts/check_cpu_runtime.py, check_control_runtime.py, check_development_e2e.py, check_combined_synthetic_e2e.py, check_desktop_control.py, check_opencode_compatibility.py, check_*kubernetes*.py, and check_remote_*e2e.py; tests/test_*.py across control, worker, desktop runners, Astra, Kubernetes, recovery, artifacts, and workflow.

Review limitations

This PDF contains rendered vector/text diagrams, not screenshots. It does not claim a live Kubernetes, OAuth, OpenCode, provider, desktop visual, or research investigation run. Test results quoted as historical are attributed to their dated repository documentation. The dirty-tree caveat matters: the report reviews working content that had not been committed and cannot establish provenance beyond the local snapshot.

Conclusion

Tessera has a coherent authority-first design and unusually explicit evidence labelling. The next quality bar is not another synthetic seam: it is one carefully bounded, independently observed, remote goal-to-decision slice. Until then, retain the accurate developer-alpha posture and resist converting plans, fixture records, or agent prose into claims of live execution, provider settlement, artifact release, or scientific validity.